SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Course Code: DSA03	Course Name: Natural Language Processing
Branch: CSE	Year : 2026

Name: R.Rohith Manikanta

Reg no:192425326

Assignment 5

1. **Write a finite-state automaton for a dialogue manager for checking your bank balance and withdrawing money at an automated teller machine.** (10M) (CO5) (BLT4)
2. **Consider the sentence "Christ opened the eyes", describes fact about English syntax.**
   a) Describe and illustrate how your favourite foreign language differs in this respect.
   b) Explain how a MT system could deal with this difference. (10M) (CO5) (BLT4)

Answer 1: Finite-state automaton for an ATM dialogue manager

A dialogue manager decides what the system says next from what the user has said so far. For a small, tightly controlled task such as an automated teller machine (ATM), it can be modelled as a **finite-state automaton (FSA)**: each state is one system prompt or action, each arc is labelled with the user input (or event) allowed in that state, and the machine follows exactly one arc per turn. This is a "system-initiative" dialogue: the machine asks one question at a time and the user can only answer that question.

Formal definition. The dialogue manager is the 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$:

- **Q** = {q0 Idle, q1 Ask PIN, q2 Main menu, q3 Show balance, q4 Ask amount, q5 Validate amount, q6 Confirm amount, q7 Dispense cash, q8 Eject card, q9 Card retained}.
- **Σ** = {card, pin_ok, pin_bad, pin_bad3, balance, withdraw, amount, amount_ok, amount_bad, yes, no, done, cancel, timeout}; each symbol is the *dialogue act* produced by the card reader, keypad or speech recogniser.
- **δ** : $Q \times \Sigma \rightarrow Q$ is the transition function (Figure 1 and Table 2). An input that is not legal in the current state is treated as a mis-recognition: the prompt is repeated and the state does not change.
- $\mathbf{q_0}$ = Idle (start state); **F** = {q8, q9}: card ejected, or card retained after three wrong PINs.

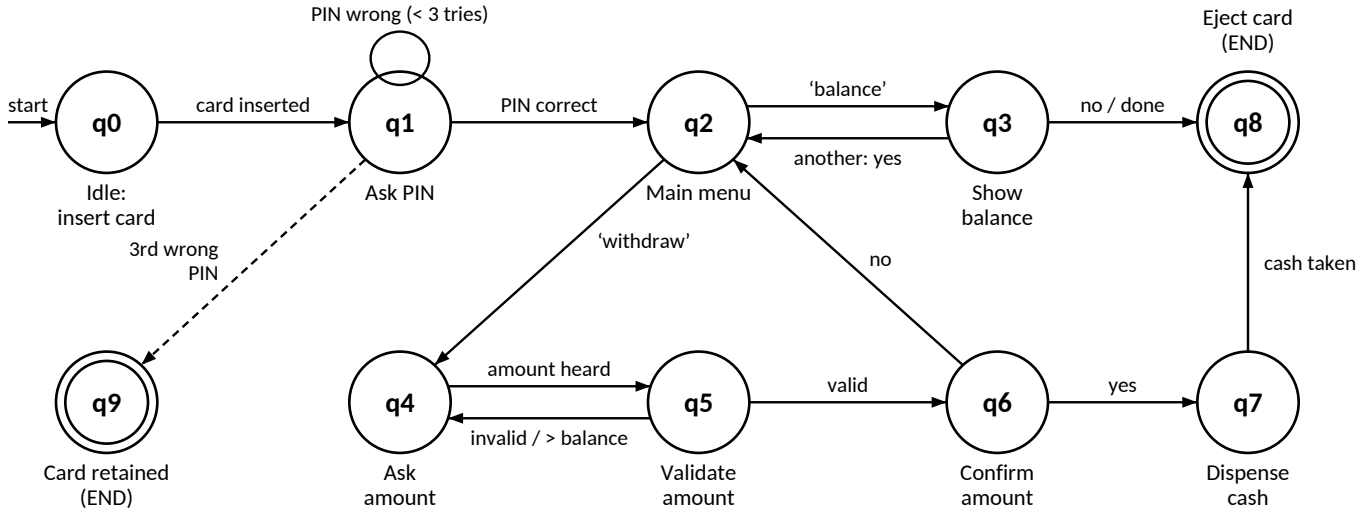
Table 1: States and the system action in each state

State	Name	System prompt / action
q0	Idle	"Please insert your card."
q1	Ask PIN	"Please enter your 4-digit PIN." (a counter records the number of tries)
q2	Main menu	"Would you like to check your balance or withdraw cash?"
q3	Show balance	Reads the balance aloud / on screen, then asks "Anything else?"
q4	Ask amount	"How much would you like to withdraw?"
q5	Validate amount	Check: multiple of note value, within daily limit, ≤ balance
q6	Confirm amount	"You asked for ₹X. Is that correct?"
q7	Dispense cash	Counts out the notes, prints the receipt, asks "Anything else?"
q8	Eject card (final)	"Please take your card. Thank you." – session ends
q9	Card retained (final)	"Your card has been retained for security." – session ends

Table 2: Transition function δ (legal moves only)

State	Input	Next	State	Input	Next
q0	card	q1	q4	amount	q5
q1	pin_ok	q2	q5	amount_ok	q6
q1	pin_bad (< 3)	q1	q5	amount_bad	q4
q1	pin_bad3	q9	q6	yes	q7
q2	balance	q3	q6	no	q2
q2	withdraw	q4	q7	done	q8
q3	yes (another)	q2	any	cancel / timeout	q8
q3	no / done	q8			

Figure 1: State-transition diagram of the ATM dialogue manager



Any state: 'cancel' / timeout → q8 (eject card). Double circle = accepting (final) state.

Circles are states, arrows are transitions labelled with the input that triggers them; the dashed arrow is the failure path after three wrong PINs.

Sample dialogue (a): withdrawal with two corrections – q0 → q1 → q1 → q2 → q4 → q5 → q4 → q5 → q6 → q7 → q8

System (q0): Please insert your card. **User:** [inserts card]

System (q1): Please enter your PIN. **User:** 4 1 2 7 (wrong)

System (q1): Incorrect PIN. Please try again. **User:** 4 1 2 9 (correct)

System (q2): Check your balance or withdraw cash? **User:** Withdraw.

System (q4): How much would you like to withdraw? **User:** ₹ 25,000.

System (q5→q4): Sorry, the daily limit is ₹ 20,000. How much? **User:** ₹ 5,000.

System (q6): You asked for ₹ 5,000. Is that correct? **User:** Yes.

System (q7): Please take your cash and receipt. **User:** [done]

System (q8): Please take your card. Thank you.

Sample dialogue (b): balance enquiry – q0 → q1 → q2 → q3 → q8

System (q3): Your balance is ₹ 48,250. Anything else? **User:** No, thanks. **System (q8):** Please take your card.

Implementation sketch. The automaton is just a lookup table plus a loop, which makes it easy to test and to audit:

```
delta = {('q0','card'):'q1',
         ('q1','pin_ok'):'q2', ('q1','pin_bad'):'q1', ('q1','pin_bad3'):'q9',
         ('q2','balance'):'q3', ('q2','withdraw'):'q4',
         ('q3','another'):'q2', ('q3','done'):'q8',
         ('q4','amount'):'q5',
         ('q5','amount_ok'):'q6', ('q5','amount_bad'):'q4',
         ('q6','yes'):'q7', ('q6','no'):'q2', ('q7','done'):'q8'}
FINAL, state = {'q8','q9'}, 'q0'
while state not in FINAL:
    act = get_input(state)                # keypad / speech act
    if act in ('cancel', 'timeout'): state = 'q8'
    else: state = delta.get((state, act), state)   # unknown input: re-prompt
    speak(prompt_for[state])
```

Design notes.

- **Verification first:** the only arc out of q0 leads to q1, so no account data is reachable before the PIN is accepted; three failures end in q9.
- **Confirmation (q6):** speech or keypad input may be wrong, so the amount is confirmed before any cash moves; invalid amounts return to q4 instead of ending the session.
- **Limits of an FSA:** the dialogue is system-initiative and rigid (the user cannot say "withdraw 5,000 rupees" in one utterance). Frame-based or information-state managers are more flexible, but for a safety-critical ATM the predictability of an FSA is an advantage.

Answer 2: "Christ opened the eyes" – a fact about English syntax

The fact. English is an **SVO** language with **fixed word order**. In "Christ opened the eyes" the subject *Christ* comes before the verb *opened* and the object *the eyes* comes after it. English nouns carry no case endings, so grammatical roles are shown *only by position*: "The eyes opened Christ" has a different (odd) meaning. The sentence also shows that the determiner *the* precedes its noun and that the verb is marked only for tense (*-ed*), with no gender or number agreement: VP → V NP.

(a) How Hindi (my chosen foreign language) differs

Hindi is an **SOV**, head-final language. The same sentence, romanised and glossed word by word:

Hindi (romanised)	**Isa Masih**	**ne**	**ānkhen**	**kholiin**
Gloss	Christ	ERG (case marker)	eyes (fem. plural)	opened (perfective, fem. plural)
English	Christ	–	the eyes	opened

- **Verb-final order.** The verb comes last (S–O–V): VP → NP V in Hindi but VP → V NP in English (Figure 2).
- **Case marking instead of position.** With a perfective transitive verb the subject takes the ergative postposition *ne*. Because roles are visible, word order is freer: *ānkhen Isa Masih ne kholiin* is also grammatical (with different emphasis), while "The eyes Christ opened" is marked in English.
- **Agreement with the object.** After *ne* the verb agrees with the *object*: *ānkhen* is feminine plural, so the verb is *kholiin*; for a masculine singular object it is *khola* (*Isa Masih ne darwaza khola*, "Christ opened the door"). English verbs show no such agreement.
- **No articles; postpositions.** Hindi has no *the / a* (definiteness comes from context), and English *in the temple* (preposition + NP) corresponds to *mandir mein* (NP + postposition).

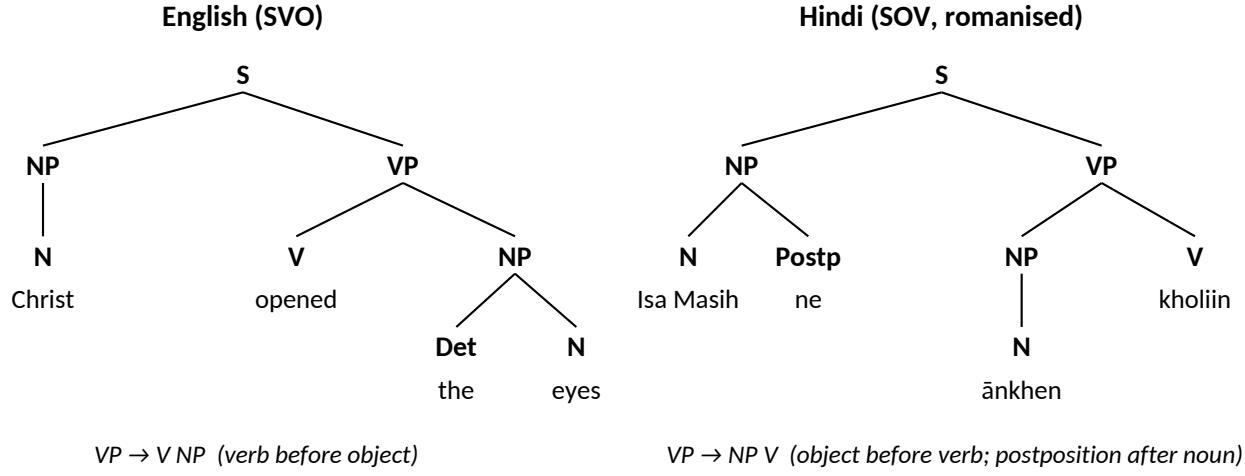


Figure 2: *Phrase-structure trees for "Christ opened the eyes" in English and in Hindi (romanised).*

Table 3: Summary of the differences

Feature	English	Hindi
Basic order	Subject – Verb – Object	Subject – Object – Verb
Adpositions	Prepositions (*in the temple*)	Postpositions (*mandir mein*)
Roles marked by	Word position	Case markers (*ne*, *ko*, *se*) plus position
Word-order freedom	Rigid	Fairly free (scrambling)
Verb agreement	Tense only (*opened*)	Gender, number, person; may agree with the object
Articles	*the*, *a* (obligatory)	None

(b) How a machine translation (MT) system can deal with the difference

English→Hindi translation must **reorder** constituents, **insert or delete** function words and **generate agreement**. Each family of MT systems does this differently.

1. Rule-based transfer MT (analysis → transfer → generation)

- **Analysis:** [S [NP Christ] [VP [V opened] [NP the eyes]]], tense = past, transitive.

- **Lexical transfer:** Christ → *Isa Masih*, open → *khol-*, eye → *ānkh* (feminine).
- **Structural transfer:** rule VP[V NP] → VP[NP V] moves the verb after the object, and *the* is deleted because Hindi has no article.
- **Insertion and agreement:** the rule "past transitive → mark the subject with *ne*" inserts the postposition, and the generator copies gender and number from the object to the verb: *Isa Masih ne ānkhen kholiin.*

The reverse direction (Hindi→English) undoes these steps and must also **insert** an article, guessed from context.

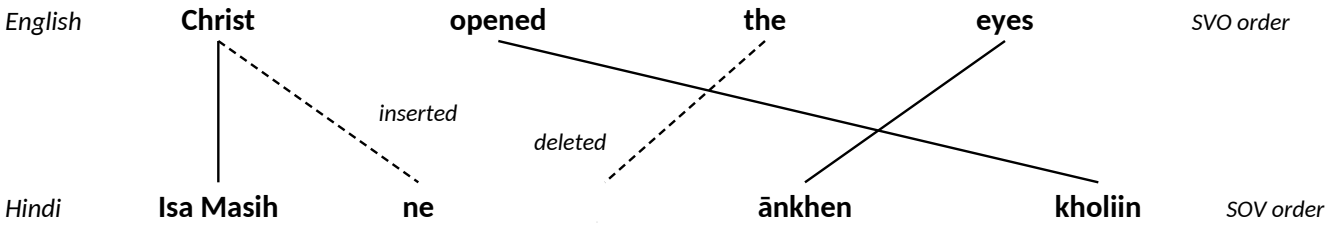


Figure 3: *Word alignment for the transfer example. Crossing lines show reordering, the dashed arrows show an inserted word (ne) and a deleted word (the).*

2. Statistical MT

- Phrase-based SMT learns phrase pairs such as *the eyes* ↔ *ānkhen* from word-aligned parallel text. Long-distance verb movement is hard, so it uses a **distortion / lexicalised reordering model** and a target language model that prefers verb-final Hindi.
- **Pre-ordering:** the English input is parsed and rearranged into Hindi-like order ("Christ the eyes opened") before translation, so alignment is almost monotone. **Syntax-based SMT** (e.g. hierarchical rule $X \rightarrow \langle V\ X_1,\ X_1\ V \rangle$) builds the reordering into the grammar.

3. Neural MT (encoder–decoder with attention / Transformer)

- Self-attention lets the decoder look at any source position, so the verb can be generated last without explicit rules; the model learns SVO→SOV reordering, the insertion of *ne* and gender/number agreement from data, with subword (BPE) units for Hindi morphology. Errors remain for rare words and long sentences.

Evaluation and challenges. Translation quality is judged with automatic metrics such as BLEU (n-gram overlap with a reference translation) and with human ratings of adequacy and fluency. Word-order mistakes, for example leaving the verb in its English position, lower both scores. Remaining difficulties are ambiguous words, names and transliteration (*Christ* → *Isa Masih*), and language pairs with little parallel data.

Conclusion. The languages differ in word order, adpositions, case marking, agreement and articles. Transfer systems use explicit rules, statistical systems use reordering and syntax-based models, and neural systems learn all of it from data, giving *Isa Masih ne ānkhen kholiin*.

Answer 1 (continued): Detailed working of the ATM dialogue manager

The ATM example is a task-oriented dialogue because the user and the machine have a specific goal: complete a balance enquiry or a cash withdrawal. The finite-state automaton keeps the conversation within a known sequence of steps. At any moment, the current state tells the dialogue manager what information is expected and what response should be presented. The next state is selected only after the system receives an input that is valid for the current state.

Meaning of the main stages

Starting the session. State q0 waits for a card-insertion event. Once the card reader reports that a card is present, the automaton moves to q1. This separation is useful because it prevents the system from showing account-specific information before a card has been introduced and the authentication process has begun.

Authentication. State q1 asks for the PIN and records the number of failed attempts. A correct PIN moves the dialogue to q2, the main menu. A wrong PIN below the retry limit leaves the machine in q1 and produces a clear retry prompt. The third consecutive failure moves to q9, the terminal card-retained state. This path makes the security policy explicit instead of leaving it as an informal instruction.

Choosing a service. In q2, the user selects either a balance enquiry or a withdrawal. A balance request moves to q3, where the account balance is retrieved and displayed or spoken. A withdrawal request moves to q4, where the system collects the amount. When the balance has been presented, the user may return to q2 to perform another service or finish the session and move to q8.

Collecting and checking an amount. State q4 accepts the amount requested by the user. The next state q5 checks the amount against the conditions stated in the assignment: it must be a valid note-value multiple, must not exceed the daily withdrawal limit, and must not be greater than the available balance. A valid amount moves to q6. An invalid amount returns to q4 so that the user can enter a corrected value.

Confirmation and completion. State q6 repeats the amount and asks the user to confirm it. A positive response moves to q7, where the machine dispenses the notes and prints a receipt. A negative response returns to q2 so that the user can choose another action. After the cash is taken, the session proceeds to q8 and the card is ejected.

The state design therefore separates collecting information, validating it, asking for confirmation and carrying out the physical action. This separation is important in a transaction: the machine should not dispense cash merely because an amount was recognised by the keypad or speech recogniser. The confirmation state provides an additional opportunity to detect an incorrect input.

Answer 1 (continued): Reliability, security and limitations of the FSA

Handling unexpected and incomplete input

A dialogue system must account for inputs that are unclear, incomplete or unsuitable for the current question. For example, while the ATM is asking for a PIN, the user may press a menu key or leave the keypad untouched. The transition rule in the original automaton treats an unrecognised input as a reason to repeat the current prompt while keeping the same state. This is a simple recovery strategy: it does not accidentally advance the transaction, and the user is told what information is required.

A timeout is different from an ordinary recognition error. It means that no valid response has arrived within the allowed interval. The given automaton sends a cancel or timeout event to q8, which ends the session by ejecting the card. This avoids leaving an authenticated session active indefinitely. A real implementation would also clear temporary session data and return the machine to its idle condition after the card has been taken.

Why validation is separate from recognition

Recognition answers the question, “What amount did the user enter?” Validation answers the different question, “Is that amount allowed for this account and transaction?” These tasks should not be confused. A perfectly recognised amount can still be invalid because it exceeds the available balance or daily limit. Conversely, a valid amount may be misrecognised and require the user to enter it again. The q4 and q5 states keep these two situations distinct.

The confirmation step is another form of error control. It gives the user a chance to check the interpreted amount before the machine performs an action that affects the bank account. The message should state the amount clearly and should accept an unambiguous yes or no response. If the user says no, the machine returns to the menu rather than assuming that the transaction should continue.

Security and user experience

- **Protect private information.** The PIN should be masked on the display and should not be spoken aloud. Balance information should only be shown after successful authentication.
- **Make prompts specific.** “Enter your four-digit PIN” is more useful than a general message such as “Continue”. Error prompts should explain what kind of input is expected without revealing sensitive information.
- **Keep the user informed.** During validation, the system should indicate that it is checking the amount. During dispensing, it should tell the user when to take the cash, receipt and card.
- **Provide a safe ending.** The final state should ensure that the card is ejected or retained according to the defined security path and that the session is closed.

Advantages and limitations

An FSA is easy to understand, implement and test. Each state has a defined purpose, and each permitted input has a predictable result. This makes the model suitable for a small ATM task where safety and consistency are more important than conversational freedom. Test cases can be written for every legal transition, including a correct PIN, repeated PIN failures, a balance enquiry, a valid withdrawal, an invalid amount, cancellation and timeout.

The main limitation is rigidity. A user cannot freely combine several requests in one sentence, interrupt a prompt with an unrelated question or change the topic without following the menu. A finite-state model can also become large when many services and exceptional cases are added. More advanced dialogue managers can use frames or information states to remember several pieces of information at once. Even so, the FSA in this assignment is a clear baseline because the task is narrow and its transaction sequence is well defined.

Answer 2 (continued): Further explanation of English and Hindi syntax

Word order and grammatical roles

The sentence “Christ opened the eyes” illustrates the usual English Subject–Verb–Object order. The subject is placed before the verb, and the direct object follows the verb. In this sentence, the position of “Christ” identifies the person performing the action, while “the eyes” identifies what is affected by the action. English relies strongly on this order because ordinary nouns do not carry a visible case ending that independently identifies their grammatical role.

Hindi generally places the object before the verb, producing Subject–Object–Verb order. The example used in the assignment is “Isa Masih ne ānkhen kholiin”. The subject phrase appears first, the object appears before the verb, and the verb occurs at the end of the clause. A direct word-for-word translation into English order would therefore not preserve the normal Hindi sentence structure. The words must be arranged according to the grammar of the target language.

Case marking and the role of ne

Hindi can mark grammatical relationships with postpositions and case markers. In the perfective transitive example, “ne” marks the subject in an ergative construction. This marker helps the reader identify the agent even though the object is placed before the verb. English normally communicates the same relationship through the order of the noun phrases and does not add an equivalent marker after “Christ”.

Because grammatical roles are indicated by case marking as well as position, Hindi permits more variation in the order of some constituents than English. A constituent may be moved to give it emphasis while the case marker continues to indicate its role. This does not mean that every possible ordering is equally natural: information structure, emphasis and context still influence which ordering a speaker would choose. English, by comparison, generally depends more heavily on the conventional subject-before-verb and object-after-verb pattern.

Agreement and morphology

The Hindi example also illustrates agreement between the verb and the object in the relevant perfective construction. The noun “ānkhen” is feminine plural, and the verb form “kholiin” reflects those features. If the object were a masculine singular noun, a different verb form could be required. English “opened” does not change according to the gender or number of the object. It expresses past tense, but it does not show the same type of object agreement.

This difference matters for language processing because a translation system cannot simply replace each English word with a Hindi word and leave the order unchanged. It must identify which noun phrase is the object, determine the features needed for the Hindi verb, and generate the appropriate inflected form. If the source sentence is longer, the system must also keep track of which modifiers belong to each noun phrase so that the correct agreement information is used.

Articles and adpositions

English uses articles such as “the” and “a” before nouns. Hindi does not have direct equivalents of these obligatory articles in the same way. Definiteness is often understood from context, discourse and other expressions. For this reason, the English word “the” may have no separate word in the Hindi translation. A translation system must decide whether the meaning is already clear from context instead of mechanically inserting a Hindi word for every article.

The two languages also differ in the placement of adpositions. English commonly uses prepositions before noun phrases, as in “in the temple”. Hindi uses postpositions after the relevant noun phrase, as in “mandir mein”. This is another example of a structural difference that affects word order. A good translation must transfer the grammatical relationship, not merely copy the location of the words.

Overall, the comparison involves several linked features: basic constituent order, case marking, agreement, articles and adpositions. These features interact. A translation system that handles only verb movement but ignores case marking or agreement may produce a sentence whose words are recognisable but whose grammar is unnatural or ambiguous.

Answer 2 (continued): How machine translation handles the differences

Rule-based transfer

A rule-based machine translation system uses explicit linguistic descriptions. First, it analyses the English sentence and identifies its constituents and grammatical relationships. For the assignment example, it recognises “Christ” as the subject, “opened” as a past-tense transitive verb and “the eyes” as the object. The transfer stage then applies language-pair rules: it selects appropriate Hindi lexical items, changes the constituent order from SVO to SOV, handles the absence of a direct article equivalent, and inserts the required case marker for the subject in the perfective construction.

The generation stage produces a grammatical Hindi sentence and selects the appropriate verb form. In the example, the object “ānkhen” is feminine plural, so the generated verb form is “kholiin”. The output is “Isa Masih ne ānkhen kholiin”. This example shows that transfer involves more than dictionary substitution. It includes lexical choice, grammatical structure, function-word handling and morphology. In the reverse Hindi-to-English direction, the system must restore English order and infer an appropriate article from context.

Statistical machine translation

A phrase-based statistical machine translation system learns translation patterns from a collection of aligned bilingual sentences. It estimates which target-language phrases are likely translations of source-language phrases and uses a target-language model to prefer fluent Hindi sequences. Since English and Hindi place the verb in different positions, the system also needs a reordering mechanism. A distortion or lexicalised reordering model estimates how phrases may move when generating the target sentence.

Pre-ordering is one way to make the task easier. The English input can first be analysed and rearranged into an order that is closer to Hindi, after which phrase translation is performed. Syntax-based statistical approaches represent some reorderings as structural rules, so the movement of a verb or phrase is captured as part of the translation model rather than treated as an arbitrary jump. These methods learn from data, but they depend on the quality and quantity of the parallel corpus.

Neural machine translation

Neural machine translation uses an encoder to represent the source sentence and a decoder to generate the target sentence. Attention mechanisms allow the decoder to use information from different positions in the source. Transformer models use self-attention to model relationships across the sentence, which helps them learn that a Hindi verb may need to be generated after the object even when the English verb appears earlier. The system can also learn when to omit an English article, when a Hindi case marker is needed, and how the verb form relates to gender and number.

Subword processing is useful for languages with rich morphology. A word may be divided into smaller units so that the model can represent stems and recurring endings even when a complete word is rare in the training data. However, neural systems can still make errors with uncommon names, ambiguous words, long-distance relationships and sentences that are poorly represented in the training corpus. A fluent-looking output is not automatically a faithful translation, so its meaning must also be checked.

Evaluation and remaining challenges

The assignment mentions BLEU, which compares the n-grams in a machine translation with those in a reference translation. It provides an automatic indication of lexical overlap, but it cannot fully judge whether the translation preserves the intended meaning. Human evaluation therefore remains important. Adequacy concerns whether the important information from the source has been preserved, while fluency concerns whether the target sentence reads naturally in its own language.

For this example, evaluation should check that the agent and the affected object have not been reversed, that the action remains in the past, that the verb appears in a natural Hindi position, and that the verb agrees appropriately with the object. It should also check that the handling of “the” does not introduce an unsupported meaning. Names and transliteration require special care because a proper name may need a conventional target-language form rather than a literal translation.

Overall, successful translation preserves the sentence meaning while adapting word order, case marking, agreement and function words to the target language.